

## Python oop.

- a) oop stand for object oriented programming.  
oop is a program method where a program is designed using classes and objects.

eg:-

```
class student:
```

```
    name="pratiksha"
```

```
    age = 23
```

```
student1 = student()
```

```
print(student1.name)
```

```
print(student1.age)
```

output

pratiksha

23

student → class

student1 → object

name, age → Attributes.

- b) class

class is a blueprint or template for creating objects.

eg:- student = student()

```
print(student.name)
```

```
print(student.age)
```

- c) object

An object is an instance of a class

eg:-

```
class student:
```

```
    name = "Ram"
```

```
student1 = student()
```

```
student2 = student()
```

## d) constructor

A constructor is a special method that is automatically called when an object is created.

`__init__()`

eg:-

```
class student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

student = student("Ram", 20)
print(student.name)
print(student.age)
```

output

Ram

20

making object

`student("Ram", 20)`

`__init__()`

automatically call

## e) self

self refers to the current object

class student:

`def __init__(self, name, age):`

`self.name = name`

`self.age = age`

`student1 = student("Ram", 20)`

`student2 = student("Sita", 21)`

`print(student1.name)`

`print(student2.name)`

output

Ram

Sita

Ram - 100  
Sita - 200  
self.name - current object name  
self.age - current object age.

f) method  
A method is a function defined inside a ~~class~~ class.

```
class student:
    def say_hello(self):
        print("Hello, I am student")

student = student()
student.say_hello()

output
Hello, I am student
```

- g) Main pillars of oop  
There are 4 main pillars are given :-
1. Encapsulation
  2. Inheritance
  3. polymorphism
  4. abstraction

## 1. Encapsulation

Encapsulation means wrapping and method together inside a class.

data & methods  
class jina rakha hai  
encapsulation hai.

class student:

def \_\_init\_\_(self, name, age):

self.name = name

self.age = age

def display(self):

print(self.name)

print(self.age)

student = student("Ram", 24)

student.display()

name, age, display inside a class

### Advantage of encapsulation

- It is easy to organize data
- data can be controlled
- It makes the code secure and manageable

## 2. Inheritance

Inheritance allows one class to use the properties and method of an other class.

parent class → base class  
child class → derived class

eg:-

```
class animal:
    def eat (self):
        print("Animal is eating")

class dog (animal):
    def bark (self):
        print("dog is barking")
```

```
dog1 = dog()
```

```
dog1.eat()
```

```
dog1.bark()
```

output

animal is eating

dog is barking

types of inheritance

there are five type of inheritance

- single inheritance
- multiple inheritance
- multiple level inheritance
- hierarchical inheritance
- hybrid inheritance

3. polymorphism

polymorphism means "many forms".

method/function of same name

object may have different behaviour

polymorphism can be

```

eg:-
class Dog:
    def sound(self):
        print("Dog barks")

class Cat:
    def sound(self):
        print("Cat meows")

dog = Dog()
cat = Cat()

dog.sound()
cat.sound()

```

output

Dog barks

Cat meows

Here both classes have the same method name, `sound()`, but they perform different tasks.

4) Abstraction

Abstraction means hiding unnecessary implementation details and showing only important information.

eg:-

when you use an ATM

- you insert your card
- you enter your pin
- you withdraw money

but you do not need to know all the details how the ATM process the money internally.

This is the concept of abstraction.

H) Method overriding  
Method overriding happens when a child class provides its own version of a method that already exists in the parent class.  
eg:-

```
class Animal:
    def Animalsound(self):
        print("Animal makes sound")

class Dog (Animal):
    def sound(self):
        print("Dog barks")

dog = Dog()
dog.sound()
output: "Dog barks"
```

parent class  $\rightarrow$  `Animal` class  
child class  $\rightarrow$  `Dog` class  
but the child class's method is executed.

9) Naming convention  
Naming convention means rules or common practices for naming variable, function, classes etc.

1. variable  $\rightarrow$  snake case  
e.g. student\_name = "Ram"  
student\_age = 20

2. Function → Snake Case

def calculate\_sum(a, b):  
 pass

3. constant → upper case

PI = 3.14

max\_value = 100

("I am a student")

4. class → Pascal Case

class StudentDetails:

"I am a student"

name = "Bob"

5. private variable → ~~lower case~~

\_student\_name = "Ram"

5) File Handling

File handling means creating, reading, writing and managing files using a program.

Basic Syntax

open("filename", mode)

common modes

mode meaning

r read

w write

a append

x create